

UNIVERSIDAD
DE GRANADA

Artificial Bee Colony

Práctica opcional

Pablo Linari Pérez

DNI: 75571079W
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del algoritmo 3
1.1 Funcionamiento del algoritmo 3
1.2 Parámetros del algoritmo 3
1.3 Ecuaciones 4
1.4 Etapas del algoritmo 4
1.4.1 Etapa 1: Inicialización 4
1.4.2 Etapa 2: Fase de abejas empleadas 4
1.4.3 Etapa 3: Fase de abejas observadoras 5
1.4.4 Etapa 4: Fase de abeja exploradora 5
1.5 Pseudocódigo 5
1.6 Adaptaciones al problema del portfolio 6
2 Mejoras del algoritmo 7
2.1 Hibridación con búsqueda local (Solis-Wets) 7
2.1.1 La búsqueda local: Solis-Wets 7
2.1.2 Integración en el ABC 7
2.2 Hibridación con Differential Evolution 8
2.2.1 Tipo 1: DE como operador de generación de candidatos 8
2.2.2 Tipo 2: DE como algoritmo completo intercalado 9
3 Propuestas de mejora 10
3.1 Vuelos de Levy 10
3.1.1 Generación del paso: algoritmo de Mantegna 10
3.1.2 Aplicación en el ABC 11
3.2 Comparación de resultados 11

1 Descripción del algoritmo

El algoritmo **Artificial Bee Colony** (ABC), propuesto por Dervis Karaboga en 2007, es una metaheurística poblacional inspirada en el comportamiento de búsqueda de alimento de las colonias de abejas. La idea del algoritmo es que un conjunto de abejas explora el espacio de búsqueda en el que hay una serie de fuentes de alimentos, intensificando la exploración alrededor de las fuentes más prometedoras y abandonando aquellas que se agotan.

En el contexto de este problema cada **fuente de alimento** representa una solución candidata, y la cantidad de néctar de una fuente se corresponde con el valor de fitness de esa solución. El objetivo del algoritmo es, por tanto, encontrar la fuente con mayor cantidad de néctar, es decir, la solución que maximiza la función objetivo.

==Explicación de los componentes del algoritmo

La colonia se divide en tres tipos de abejas, cada uno con un papel distinto dentro de la búsqueda:

- **Abejas obreras (worker bees)**: cada una está asociada a una fuente de alimento concreta y se encarga de explotarla buscando soluciones mejores en su vecindario.
- **Abejas observadoras (onlooker bees)**: esperan en la colmena y eligen qué fuentes explotar en función de la información que comparten las obreras. Las fuentes con más néctar tienen más probabilidad de ser elegidas, esto hace que se concentre la búsqueda en las funetes de alimento más prometedoras.
- **Abejas exploradoras (scout bees)**: cuando una fuente se agota se abandona, y la abeja obrera que trabajaba sobre esta se convierte en exploradora, buscando una fuente de alimento nueva.

El número de fuentes de alimento, según se indica en el artículo, es la mitad del tamaño total de la colonia, y coincide tanto con el número de abejas trabajadoras como con el de abejas observadoras ya que cada fuente tiene una trabajadora asociada.

Esta forma de inicializar la población es la propuesta en el artículo , no obstante veremos que se puede modificar este parámetro para cambiar el comportamiento del algoritmo aunque no sea el parámetro que más influya.

1.1 Funcionamiento del algoritmo

1.2 Parámetros del algoritmo

El comportamiento del algoritmo se controla mediante los siguientes parámetros:

- **colonySize**: tamaño total de la colonia. El número de fuentes de alimento se calcula como $SN = \frac{\text{colonySize}}{2}$. En la implementación se usa un valor de 50, por lo que $SN = 25$.
- **Limit** : número máximo de ciclos que una fuente puede permanecer sin mejorar antes de ser abandonada. Controla cuanto se pueden explotar las soluciones.
- **maxevals**: número máximo de evaluaciones de la función objetivo, que actúa como criterio de parada.

1.3 Ecuaciones

El algoritmo usa dos ecuaciones . La primera genera una solución candidata v_i a partir de una fuente existente x_i, perturbando una única componente j elegida al azar:

$$v_{ij} = x_{ij} + \phi_{ij} \cdot (x_{ij} - x_{kj})$$

donde $k \neq i$ es otra fuente vecina elegida aleatoriamente y ϕ_{ij} es un número aleatorio en el intervalo $[-1, 1]$. El resto de componentes de v_i se copian directamente de x_i.La nueva solución se acerca o se aleja de una fuente vecina en una sola dimensión.

Para entenderlo mejor , este movimiento en el problema del portfolio equivale a tomar una solución i , y sumar o resar a una empresa de dicha solución un porcentaje de la misma empresa pero de la otra solución escogida k.De esta forma nos acercaríamos o alejaríamos de la solución escogida en dicha componente.

La segunda ecuación calcula la probabilidad de que cada fuente sea seleccionada por una observadora, de forma proporcional a su néctar:

$$p_i = \frac{\text{fit}_i}{\sum_{n=1}^{SN} \text{fit}_n}$$

Cuanto mayor es el fitness de una fuente, mayor es su probabilidad de ser explotada por las observadoras.

1.4 Etapas del algoritmo

La ejecución se divide en 4 fases claras donde se identifica el comportamiento de cada abeja con su rol.

1.4.1 Etapa 1: Inicialización

Se generan SN fuentes de alimento de forma aleatoria, asegurando que cada una cumple las restricciones del problema (en el caso del portfolio se comprueban las restricciones necesarias). Se evalúa el fitness de cada una y se memoriza la mejor solución encontrada hasta el momento. Además, se inicializa a 0 un contador de *trials* por cada fuente, que registra cuántos ciclos lleva sin mejorar.

1.4.2 Etapa 2: Fase de abejas empleadas

Cada abeja empleada explota su propia fuente x_i. Para ello genera una solución candidata v_i aplicando la ecuación (2.2) y la evalúa. A continuación se aplica una selección greedy: si la candidata es mejor que la fuente original, la sustituye y el contador de *trials* de esa fuente se reinicia a 0, en caso contrario, la fuente se mantiene y su contador de *trials* se incrementa. Esta fase genera por cada fuente de alimento una posible solución mejor usando la ecuación previamente vista, si esta solución es mejor será la que se use como base para la explotación.

1.4.3 Etapa 3: Fase de abejas observadoras

Esta etapa consiste en el «baile de las obreras» donde cada obrera vuelve a la colmena e intentará convencer a las observadoras de elegir su fuente de alimento , las observadoras eligirán con mayor probabilidad la fuente que más nectar tenga.Los pasos que se siguen una vez la obrera vuelve a la colmena son los siguientes. Primero se calculan las probabilidades de selección de cada fuente con la segunda ecuación . Después, se reparten SN observadoras: cada una elige una fuente mediante selección por ruleta ,(las fuentes con más néctar tienen más probabilidad) y la explota igual que una empleada, usando la misma ecuación y aplicando de nuevo la selección greedy. Como las mejores fuentes tienden a ser elegidas varias veces, esta fase intensifica la búsqueda en las regiones más prometedoras del espacio. Es decir que podemos tener varias abejas observadoras en la misma fuente de alimento, esto ocurrirá sobre todo en las fuente más prometedoras donde como es lógico, interesa intensificar la búsqueda.

1.4.4 Etapa 4: Fase de abeja exploradora

Al final de cada ciclo se comprueba si alguna fuente se ha agotado. Se localiza la fuente con mayor número de *trials* acumulados y, si este supera el *Limit*, dicha fuente se abandona, se sustituye y la abeja exploradora buscaría una nueva, que a modo práctico consiste en crear una nueva solución aleatoria,se evalúa la nueva solución y su contador de *trials* vuelve a 0. Se permite como máximo una abeja exploradora por ciclo. Esta fase introduce diversificación y evita que el algoritmo quede estancado en óptimos locales.

En las tres fases se actualiza siempre la mejor solución global encontrada, que es la que finalmente devuelve el algoritmo.

1.5 Pseudocódigo

```
funcion ABC(problema, maxevals):
  SN = colonySize / 2
  # --- Inicialización ---
  para i en 0..SN:
    fuentes[i]   = problema.createSolution()
    fitness[i]   = problema.fitness(fuentes[i])
    trials[i]    = 0
    actualizar_mejor(fuentes[i], fitness[i])

  # --- Ciclos de búsqueda ---
  mientras evals < maxevals:

    # (a) Fase empleadas
    para i en 0..SN:
      candidata = produceCandidate(fuentes, i)   # ec. 2.2
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)
```

```
    # (b) Fase observadoras
    probs = computeProbabilities(fitness)         # ec. 2.1
    para t en 0..SN:
      i = rouletteSelect(probs)
      candidata = produceCandidate(fuentes, i)    # ec. 2.2
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)

    # (c) Fase exploradora (máx. una por ciclo)
    peor = indice con mayor trials
    si trials[peor] >= trialLimit:
      fuentes[peor] = problema.createSolution()
      trials[peor]  = 0
      actualizar_mejor(fuentes[peor])

  devolver mejor_solucion
```

1.6 Adaptaciones al problema del portfolio

La formulación original de ABC está pensada para la optimización de funciones numéricas sin restricciones, por lo que ha sido necesario realizar dos adaptaciones:

- **Reparación de soluciones**: la ecuación que se usa para explorar nuevas soluciones modifica una componente y, en general, rompe las restricciones del problema (los pesos deben sumar 1 y estar en $[\text{low}, \text{high}]$ o ser 0). Por ello, tras generar cada candidata se comprueba su validez y, si no es factible, se repara con `problem.fix()` antes de evaluarla.
- **Desplazamiento de las probabilidades**: la ecuación (2.1) asume cantidades de néctar no negativas, pero el fitness del portfolio puede ser negativo, de hecho en la práctica es siempre negativo. Para poder aplicar la ruleta, se desplazan todos los valores de fitness sumándoles una constante que deja el peor de ellos ligeramente por encima de 0, conservando las diferencias relativas entre fuentes.

Por lo que respecta a la implementación , la diferencia entre la implementación para el problema del portfolio y para el **cec2017** son las mencionadas previamente y el uso de las funciones de la clase **pPortfolio** , principalmente usadas para poder arreglar y generar soluciones con las restricciones del problema.

2 Mejoras del algoritmo

2.1 Hibridación con búsqueda local (Solis-Wets)

El ABC clásico es un buen explorador del espacio de búsqueda, pero su capacidad de **refinamiento local** es limitada: la ecuación (2.2) solo modifica una componente por candidata y depende de las diferencias entre fuentes, por lo que cuando la población converge esas diferencias se hacen pequeñas y el avance se ralentiza. La idea de esta hibridación es convertir el algoritmo en un **algoritmo memético**, combinando la exploración global del ABC con una **búsqueda local** que pula con detalle las soluciones más prometedoras.

2.1.1 La búsqueda local: Solis-Wets

Solis-Wets es una búsqueda local estocástica de paso adaptativo que opera sobre **una única solución**. En lugar de explorar todo el espacio, da pequeños pasos aleatorios alrededor de la solución actual y aprende en qué dirección conviene moverse mediante dos elementos:

- Un **paso** δ que controla el tamaño de las perturbaciones y que se ajusta de forma dinámica según el algoritmo vaya teniendo éxito o no.
- Un vector de **sesgo** (*bias*) que acumula la dirección de los movimientos que han funcionado, de modo que la búsqueda tiende a continuar avanzando hacia donde ya ha mejorado.

En cada iteración se genera una perturbación aleatoria $d \sim U(0, \delta)$ y se construye una solución candidata sumando el paso y el sesgo:

$$x' = x + d + \text{bias}$$

Si la candidata mejora, se acepta y se **refuerza** el sesgo en esa dirección. Si no mejora, se prueba el **movimiento opuesto** $x' = x - d - \text{bias}$ (aprovechando la misma perturbación en sentido contrario); si este mejora, se acepta y el sesgo se ajusta en consecuencia. Si ninguno de los dos mejora, se considera un fracaso y el sesgo se reduce a la mitad para «olvidar» una dirección que ya no es útil.

El paso δ se adapta para equilibrar exploración y precisión: tras **5 éxitos consecutivos** se duplica (la búsqueda va bien, conviene avanzar más rápido) y tras **3 fracasos consecutivos** se divide entre dos (hay que afinar con pasos más cortos). Este mecanismo permite que la BL dé pasos grandes cuando está lejos del óptimo local y pasos finos cuando está cerca de él.

2.1.2 Integración en el ABC

La búsqueda local no sustituye a ninguna fase del ABC, sino que se **intercala** dentro del bucle principal. La hibridación se controla con tres parámetros:

- **ls_freq**: cada cuántos ciclos del ABC se lanza la búsqueda local (se usa 50).
- **ls_budget**: número de evaluaciones que se concede a cada aplicación de la BL (se usa 500).

- **ls_delta**: paso inicial δ de Solis-Wets (se usa 0.2).

Al final de cada ciclo, y solo una vez cada `ls_freq` ciclos, se localiza la **mejor fuente** de la población actual y se le aplica Solis-Wets con un presupuesto de `ls_budget` evaluaciones. Después se reinicia su contador de *trials* (acaba de ser intensificada, no tiene sentido considerarla agotada). Las evaluaciones que consume la BL se descuentan del presupuesto global del algoritmo, de forma que ABC y BL **comparten el mismo límite de evaluaciones**: la búsqueda local no es «gratis», sino que se le roban evaluaciones a la fase poblacional.

De este modo el ABC se encarga de la **diversificación** (encontrar regiones prometedoras del espacio) y Solis-Wets de la **intensificación** (exprimir al máximo la mejor solución encontrada), repartiendo el esfuerzo entre ambos a lo largo de la ejecución.

```
# dentro del bucle principal del ABC, al final de cada ciclo:
cycle += 1
si use_ls y cycle % ls_freq == 0 y evals < maxevals:
  best = indice de la fuente con menor obj    # la mejor
  budget = min(ls_budget, maxevals - evals)   # no pasarse del limite
  evals += soliswets(foods[best], obj[best], ls_delta, budget, ...)
  trial[best] = 0                             # recien intensificada
  actualizar_mejor(foods[best])
```

2.2 Hibridación con Differential Evolution

Differential Evolution (DE) es una metaheurística poblacional que genera nuevas soluciones combinando las **diferencias entre individuos** de la población. Su operador básico es la **mutación diferencial**: a un vector base se le suma la diferencia escalada entre otros dos individuos, $F \cdot (x_{r2} - x_{r3})$, donde F es un factor de escala. Después, un **cruce binomial** mezcla el mutante con la solución original copiando cada componente con probabilidad CR, garantizando con un índice aleatorio (`jrand`) que al menos una componente provenga del mutante. Finalmente se aplica una selección *greedy* entre la candidata y la original.

A diferencia de la ecuación (2.2) del ABC, que solo modifica una componente y usa una única fuente vecina, el operador de DE **combina información de toda la población en varias dimensiones a la vez**, lo que suele dar un mejor poder de búsqueda. Se han probado **dos formas distintas** de incorporar DE al ABC.

2.2.1 Tipo 1: DE como operador de generación de candidatos

En esta primera variante DE **no se añade como una fase nueva**, sino que **reemplaza la regla de movimiento** de las abejas. Donde el ABC clásico usaba la ecuación (2.2), ahora se usa el operador **DE/current-to-best/1** con cruce binomial. Para cada componente seleccionada por el cruce:

$$v_{ij} = x_{ij} + F \cdot \left(x_{\text{best } j} - x_{ij}\right) + F \cdot \left(x_{r1j} - x_{r2j}\right)$$

donde x_{best} es la mejor fuente actual del enjambre y $r1, r2$ son dos índices aleatorios distintos entre sí, de i y de *best*. El término $F \cdot (x_{\text{best}} - x_i)$ atrae la candidata hacia la mejor solución (intensificación dirigida), mientras que $F \cdot (x_{r1} - x_{r2})$ añade una perturbación

diferencial que mantiene la diversidad. El cruce binomial con probabilidad CR decide qué componentes se mutan, y `jrand` asegura que al menos una cambie.

Este operador se usa **en las dos fases de explotación** del ABC (abejas empleadas y observadoras), de modo que el esqueleto del algoritmo (ruleta de las observadoras, exploradoras según el *Limit*, criterio de parada) se mantiene intacto y la única diferencia con el ABC original es **cómo se genera cada candidata**. Los parámetros usados son $F = 0.5$ y $CR = 0.9$, valores habituales en DE.

```
# reemplaza a produceCandidate (ec. 2.2) en empleadas y observadoras:
funcion produceCandidate_DE(foods, i):
  best = indice de la mejor fuente
  r1, r2 = dos indices aleatorios distintos de i, best y entre si
  jrand = dimension aleatoria
  cand = copia(foods[i])
  para j en 0..dim:
    si j == jrand o rand() < CR:                        # cruce binomial
      cand[j] = foods[i][j] + F*(foods[best][j]-foods[i][j])
                            + F*(foods[r1][j]-foods[r2][j])
  devolver clip(cand)
```

2.2.2 Tipo 2: DE como algoritmo completo intercalado

La segunda variante sigue **exactamente el mismo esquema que la hibridación con Solis-Wets**: en lugar de modificar la generación de candidatos, se **intercala un DE completo** dentro del bucle del ABC. Aquí DE no es un operador, sino un **algoritmo con su propio bucle generacional**: recorre toda la población aplicando a cada individuo mutación, cruce y selección *greedy*, y repite el proceso hasta agotar su presupuesto de evaluaciones. La variante usada es **DE/rand/1/bin**, cuya mutación parte de un vector base aleatorio en lugar de la mejor fuente:

$$v_{ij} = x_{r1j} + F \cdot \left(x_{r2j} - x_{r3j}\right)$$

con $r1, r2, r3$ tres índices distintos entre sí y de i. Al partir de un individuo aleatorio en vez del mejor, esta versión es **más exploradora** que la del Tipo 1.

DE cumple aquí el papel que tenía Solis-Wets en la hibridación memética: opera sobre las fuentes del ABC, las modifica *in place* y devuelve el número de evaluaciones consumidas para que el ABC las descuente de su presupuesto. Se controla con los mismos tres parámetros:

- **de_freq**: cada cuántos ciclos del ABC se lanza el DE (se usa 50).
- **de_budget**: evaluaciones concedidas a cada ejecución del DE (se usa 500).
- **F, CR**: parámetros del DE ($F = 0.5$, $CR = 0.9$).

La diferencia conceptual entre las dos variantes es clara: en el **Tipo 1** el comportamiento de DE está presente **en todo momento** (sustituye al movimiento de las abejas), mientras que en el **Tipo 2** el ABC y el DE se **alternan por fases**, dejando que cada uno trabaje sobre la población durante un número de evaluaciones antes de devolver el control al otro. El Tipo 2 permite además aislar el efecto de DE, ya que activarlo o desactivarlo no cambia el resto del algoritmo.

```
# algoritmo DE completo, ejecutado sobre la poblacion del ABC:
funcion differential_evolution(foods, obj, trial, F, CR, maxevals):
  evals = 0
  mientras evals < maxevals:
    para i en 0..SN:
      r1, r2, r3 = tres indices distintos de i y entre si
      jrand = dimension aleatoria
      cand = copia(foods[i])
      para j en 0..dim:
        si j == jrand o rand() < CR:                  # cruce binomial
          cand[j] = foods[r1][j] + F*(foods[r2][j]-foods[r3][j])
      candfit = fitness(clip(cand));  evals += 1
      si candfit < obj[i]:                            # seleccion greedy
        foods[i] = cand;  obj[i] = candfit;  trial[i] = 0
  devolver evals

# dentro del bucle principal del ABC, al final de cada ciclo:
cycle += 1
si use_de y cycle % de_freq == 0 y evals < maxevals:
  budget = min(de_budget, maxevals - evals)
  evals += differential_evolution(foods, obj, trial, F, CR, budget)
```

3 Propuestas de mejora

3.1 Vuelos de Levy

Una de las mejoras que se pueden aplicar consiste en cambiar la forma en la que las abejas generan nuevas soluciones. En el ABC clásico el movimiento que define la ecuación (2.2) es **local y acotado**: la perturbación $\phi_{ij} \cdot (x_{ij} - x_{kj})$ produce pasos de tamaño parecido y, como $\phi \in [-1, 1]$, nunca genera un salto grande. Esto explora bien en las cercanías de una fuente, pero hace que el algoritmo tenga dificultades para escapar de óptimos locales.

Los **vuelos de Levy** sustituyen ese movimiento por una caminata aleatoria en la que la longitud de cada paso sigue una distribución de **cola pesada**: la mayoría de los pasos son pequeños (lo que permite un refinamiento local fino), pero de vez en cuando aparece un salto muy largo capaz de sacar a la abeja de la cuenca de atracción en la que está atascada. Esta combinación de pasos cortos y saltos largos esporádicos mejora la exploración global del espacio de búsqueda.

3.1.1 Generación del paso: algoritmo de Mantegna

No existe una fórmula cerrada sencilla para muestrear directamente una distribución de Levy estable, por lo que se utiliza el **método de Mantegna**, que aproxima un paso de Levy a partir de dos variables aleatorias normales. El paso se calcula como:

$$\text{paso} = \frac{u}{|v|^{\frac{1}{\beta}}}, \quad u \sim N(0, \sigma_u^2), \quad v \sim N(0, 1)$$

donde $\beta \in (0, 2]$ es el exponente de la distribución (se usa $\beta = 1.5$) y la desviación σ_u se obtiene una única vez, ya que es constante para un β fijo:

$$\sigma_u = \left(\frac{\Gamma(1+\beta) \cdot \sin(\pi\beta/2)}{\Gamma\left(\frac{1+\beta}{2}\right) \cdot \beta \cdot 2^{\frac{\beta-1}{2}}} \right)^{\frac{1}{\beta}}$$

siendo Γ la función gamma. La cola pesada surge del cociente $u/\ |v|^{1/\beta}$: cuando v toma un valor cercano a 0, el denominador se hace minúsculo y el paso se dispara, generando el salto largo ocasional; el resto del tiempo v tiene un tamaño normal y el paso resultante es pequeño.

3.1.2 Aplicación en el ABC

La regla de generación del candidato (ecuación 2.2) se reemplaza por un paso de Levy dirigido hacia la mejor fuente del enjambre. Sobre la componente j elegida al azar:

$$v_{ij} = x_{ij} + \alpha \cdot \text{paso} \cdot \left(x_{ij} - x_{\text{best } j}\right)$$

donde cada factor cumple un papel concreto:

- El término $\left(x_{ij} - x_{\text{best } j}\right)$ aporta **dirección y escala** al salto: lo orienta respecto a la mejor solución encontrada y lo hace proporcional a la distancia a la que se encuentra la abeja de ella, de modo que las fuentes lejanas dan saltos grandes y las cercanas refinan finamente.
- El factor α (con valor 0.01) es un **factor de escala** que evita que los saltos largos se salgan de forma sistemática del dominio del problema.
- Como u está centrada en 0, el paso puede ser positivo o negativo, por lo que el candidato puede acercarse o alejarse de la mejor fuente, manteniendo la diversidad de la población.

Al igual que en el ABC clásico, tras generar la candidata se recorta al dominio $[\text{lower}, \text{upper}]$ antes de evaluarla. El resto de fases del algoritmo (observadoras con ruleta, exploradoras según el *Limit* y criterio de parada) se mantienen idénticas, de forma que la única diferencia con el ABC original es la regla de movimiento, lo que permite atribuir cualquier cambio en el rendimiento exclusivamente a la incorporación de los vuelos de Levy.

3.2 Comparación de resultados

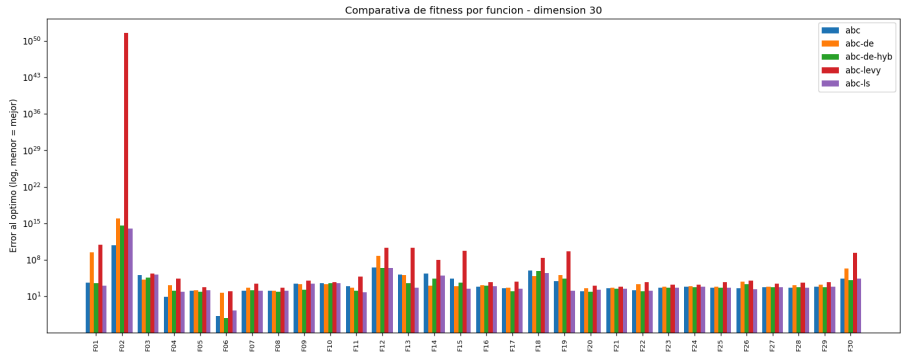


Figura 1: Comparación error para dimensión 30

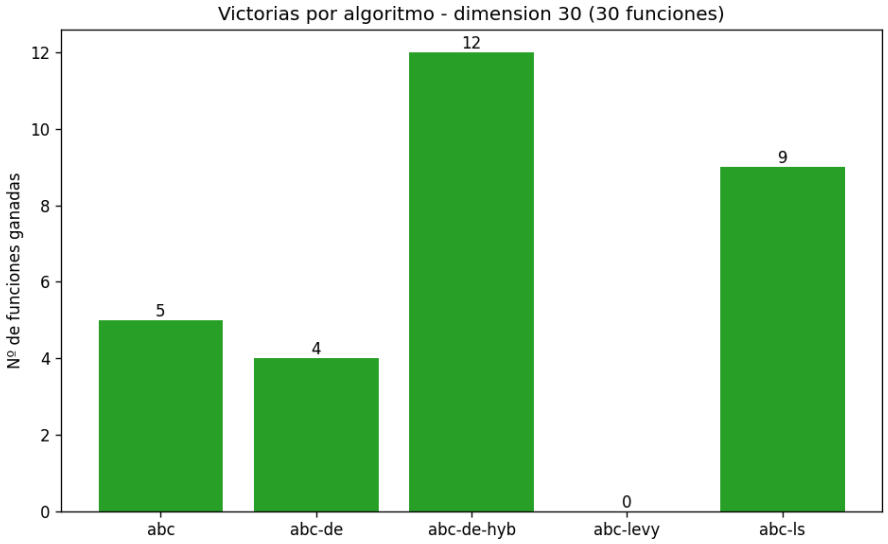


Figura 2: Número de victorias dimensión 30

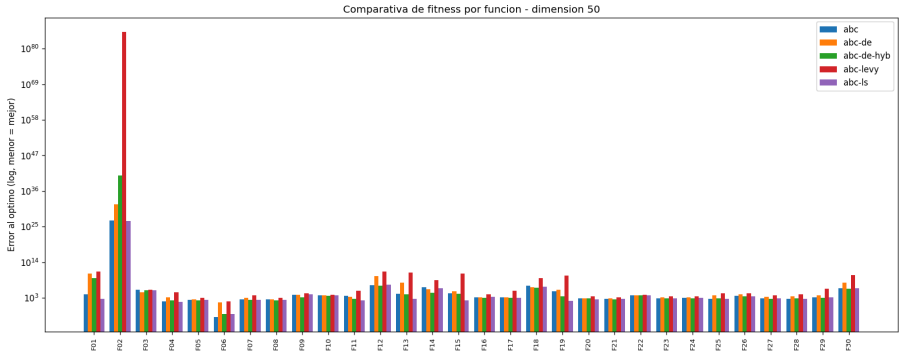


Figura 3: Comparación error para dimensión 50

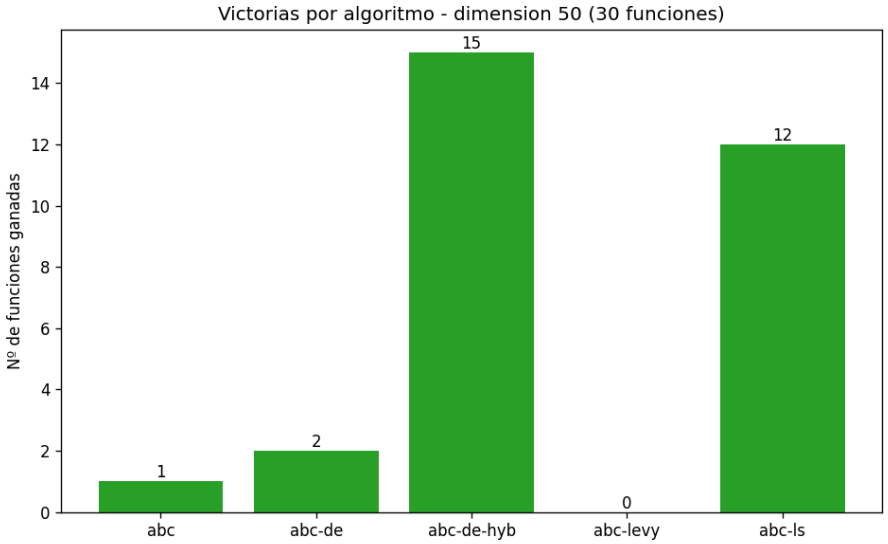


Figura 4: Número de victorias dimensión 50

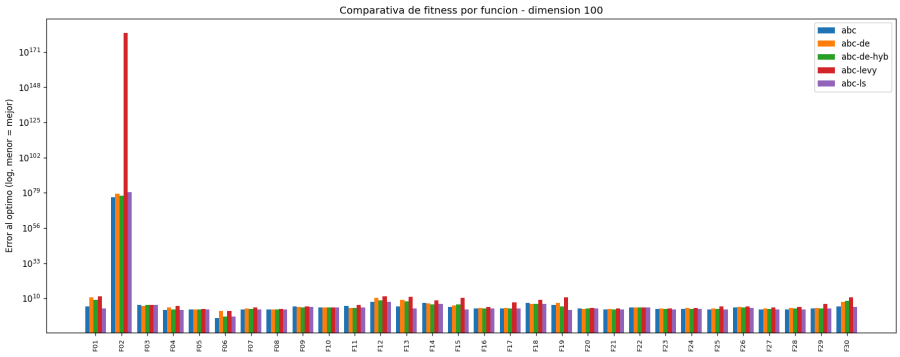


Figura 5: Comparación error para dimensión 100

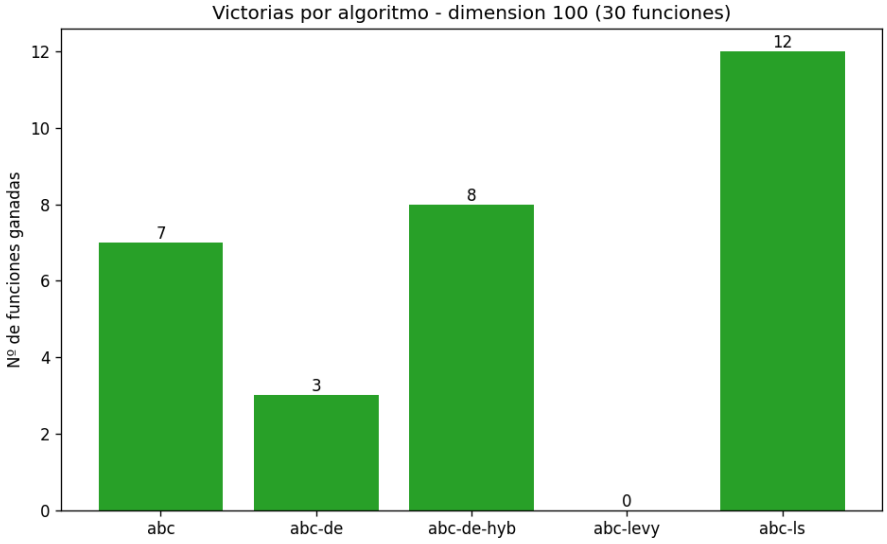


Figura 6: Número de victorias dimensión 100